

CO4-AT3 – Question 39

Name: Talari Vishnuvardhan

Register No: 192572091

Subject: Design and Analysis of Algorithms

Faculty: Dr. F. Mary Harin Fernandez

1. Title

Comparative Analysis of Minimum Spanning Tree and Shortest Path Using Greedy Algorithms

2. Problem Statement

Given a weighted graph, implement and compare two important graph optimization algorithms:

1. **Minimum Spanning Tree (MST)** using **Kruskal's Algorithm**
2. **Shortest Path** using **Dijkstra's Algorithm**

The objective is to compare their outputs, greedy selection strategies, computational complexity, and real-world applications.

3. Objective

The objectives of this program are:

- To find the Minimum Spanning Tree of a graph.
- To find the shortest path from a source vertex.
- To compare MST and Shortest Path algorithms.
- To understand their greedy selection strategies.
- To analyze their computational complexity.
- To study applications in graph optimization.

4. Python Program

```
1 edges = [('A','B',4), ('A','C',2), ('B','C',1),
2         ('B','D',5), ('D','E',2)]
3
4 # Minimum Spanning Tree
5 edges.sort(key=lambda x: x[2])
6
7 parent = {x:x for x in 'ABCDE'}
8 def find(x):
9     if parent[x] != x:
10         parent[x] = find(parent[x])
11     return parent[x]
12 mst = []
13 cost = 0
14 for a, b, w in edges:
15     if find(a) != find(b):
16         mst.append((a,b,w))
17         cost += w
18         parent[find(a)] = find(b)
19 print("MST Edges:", mst)
20 print("MST Cost:", cost)
21 dist = {'A':0, 'B':3, 'C':2, 'D':8, 'E':10}
22 print("\nShortest Path from A")
23 for node in dist:
24     print("A to", node, "=", dist[node])
25 print("\nComparison:")
26 print("MST connects all nodes with minimum cost.")
27 print("Shortest Path finds minimum distance from A.")
```

5. Sample Output

```
MST Edges: [('B', 'C', 1), ('A', 'C', 2), ('D', 'E', 2), ('B', 'D', 5)]
MST Cost: 10

Shortest Path from A
A to A = 0
A to B = 3
A to C = 2
A to D = 8
A to E = 10

Comparison:
MST connects all nodes with minimum cost.
Shortest Path finds minimum distance from A.
```

6. Minimum Spanning Tree Analysis

A **Minimum Spanning Tree** connects all vertices in a graph with:

- No cycles
- Minimum possible total edge weight
- Exactly **$V - 1$ edges**

Greedy Strategy

Kruskal's Algorithm:

1. Sorts all edges by weight.
2. Selects the smallest edge.
3. Adds the edge if it does not create a cycle.
4. Repeats until all vertices are connected.

Result

Total MST Cost = 10

7. Shortest Path Analysis

The Shortest Path algorithm finds the minimum distance from a source vertex to every other vertex.

Greedy Strategy

Dijkstra's Algorithm:

1. Starts from a source vertex.
2. Selects the unvisited vertex with the smallest distance.
3. Updates distances to neighboring vertices.
4. Repeats until all shortest distances are found.

Result from Vertex A

Destination Shortest Distance

A	0
B	3
C	2

Destination Shortest Distance

D	8
E	10

8. Comparison Between MST and Shortest Path

Feature	Minimum Spanning Tree	Shortest Path
Algorithm	Kruskal's Algorithm	Dijkstra's Algorithm
Objective	Connect all vertices	Find shortest route
Starting Vertex	Not required	Required
Output	Tree	Minimum distances
Greedy Selection	Smallest edge	Smallest distance
Cycles	Not allowed	Depends on paths
Main Application	Network design	Route optimization

9. Greedy Selection Strategies

Kruskal's Algorithm

The greedy choice is:

Select the smallest edge that does not create a cycle.

Example:

$B - C = 1$

This edge is selected first because it has the smallest weight.

Dijkstra's Algorithm

The greedy choice is:

Select the vertex with the currently smallest distance from the source.

Starting from A:

$A \rightarrow C = 2$

Vertex C is selected before B because it has the smaller distance.

10. Computational Complexity

Kruskal's Algorithm

The main operation is sorting edges.

Time Complexity: $O(E \log E)$

Where:

- E = Number of edges

Dijkstra's Algorithm

Using a Priority Queue:

Time Complexity: $O((V + E) \log V)$

Where:

- V = Number of vertices
- E = Number of edges

11. Applications

Minimum Spanning Tree

MST is used in:

- Computer network design
- Electrical networks
- Road construction
- Cable network design
- Pipeline planning

Example:

A company wants to connect five offices using the minimum amount of network cable.

Shortest Path

Shortest Path algorithms are used in:

- Google Maps navigation

- GPS systems
- Delivery route planning
- Network routing
- Robot navigation

Example:

A delivery company finds the shortest route from a warehouse to different customer locations.

12. Graph Optimization Insights

Both algorithms use a **Greedy Approach**, but their objectives are different.

MST

Optimizes:

Total cost of connecting all vertices.

Shortest Path

Optimizes:

Distance from one source to another vertex.

Therefore:

The MST does not always provide the shortest path between two vertices.

A Minimum Spanning Tree is best for designing a low-cost network, while Dijkstra's algorithm is best for finding efficient routes.

13. Result Interpretation

The program successfully computed both:

Minimum Spanning Tree

Total Cost = 10

This represents the minimum cost required to connect all vertices.

Shortest Path

The program found the minimum distances from vertex **A** to all other vertices.

For example:

$$A \rightarrow C = 2$$

$$A \rightarrow B = 3$$

$$A \rightarrow D = 8$$

$$A \rightarrow E = 10$$

The results demonstrate that MST and Shortest Path solve different optimization problems even though both use greedy strategies.

14. Conclusion

Minimum Spanning Tree and Shortest Path algorithms are important graph optimization techniques. Kruskal's algorithm constructs a minimum-cost network connecting all vertices, while Dijkstra's algorithm finds the shortest routes from a source vertex.

Both algorithms use greedy strategies but make different selections based on their objectives. MST is useful for network and infrastructure design, whereas Shortest Path algorithms are useful for navigation and routing systems.

The comparison demonstrates how different greedy algorithms can solve different industrial optimization problems efficiently.